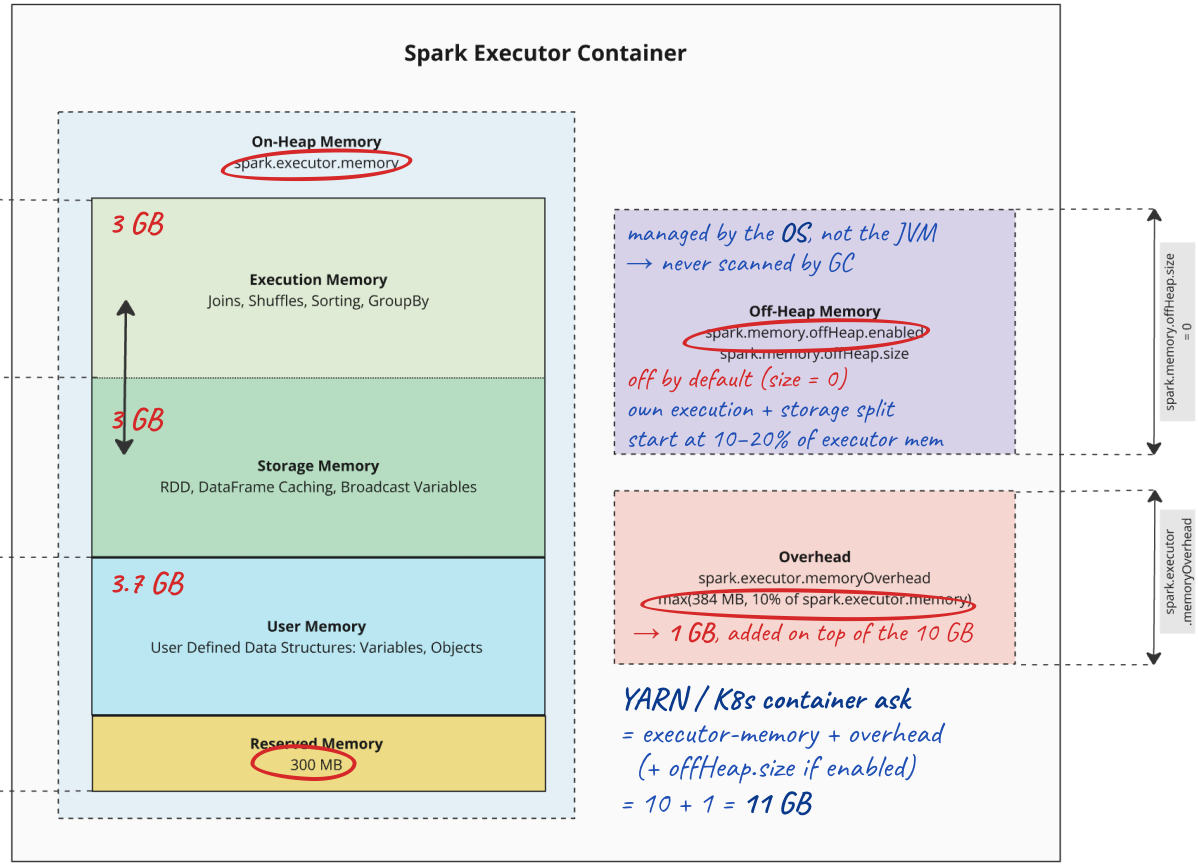
Spark Memory Management — executor layout

one executor, `--executor-memory 10G`, every default left alone · base board: `docs/images/memory_management.png`

`--executor-memory 10G`
sizes the on-heap box only
× 0.6 memory.fraction = **6 GB** unified
× 0.5 storageFraction = **3 GB** storage
6 − 3 = **3 GB** execution



10 − 6 = **4 GB** left over
4 GB − 300 MB = **3.7 GB** user memory
300 MB reserved is a flat constant — not a fraction

YARN / K8s container ask
= executor-memory + overhead
(+ offHeap.size if enabled)
= 10 + 1 = **11 GB**

💡 **`--executor-memory` sizes the heap, not the container.** That gap — overhead, plus off-heap if you enabled it — is what gets an executor killed.

✍️ What you type vs what you are charged for

you type

`--executor-memory 10G`

= the JVM heap,
and nothing else

→ YARN / K8s actually reserves a 11 GB container

- on-heap 10 GB
  5.8 unified · 3.9 user
  · 0.3 reserved
- overhead 1 GB
  off-heap would add
  a third box here

→ go past this line and the kernel kills the executor — you get no OOM trace

Reading the layout

✍️ Definition: the five pools inside one executor

pool	what actually lives there	sized by
Execution	joins, shuffles, sorts, groupBy / aggregation buffers — the scratch space of a running task	`memory.fraction × (1 − storageFraction)`
Storage	`cache()` / `persist()` blocks, broadcast variables	`memory.fraction × storageFraction`
User	your own objects: lists, dicts, lookup maps, UDF closures, anything the JVM holds that is not a Spark buffer	what is left: `(1 − memory.fraction)` − 300 MB
Reserved	Spark's own internal objects. Untouchable.	`300 MB`, hard-coded
Overhead	JVM metaspace + thread stacks, netty shuffle buffers, **PySpark worker processes**	`max(384 MB, 0.10 × executor-memory)`

💡 Execution and Storage are the only two that can lend to each other. The other three are walls.

✍️ The whole calculation, once

for `--executor-memory 10G`, defaults everywhere (take 1 GB = 1000 MB to keep the arithmetic readable):

step	arithmetic	result
on-heap total	what you typed	**10 GB**
unified	0.6 × 10	**6 GB**
↳ storage	0.5 × 6	**3 GB**
↳ execution	6 − 3	**3 GB**
user + reserved	10 − 6	**4 GB**
↳ reserved	fixed constant	**300 MB**
↳ user	4 GB − 300 MB	**3.7 GB**
overhead	max(384 MB, 10% of 10 GB)	**1 GB**
container ask	10 + 1 + 0	**11 GB**

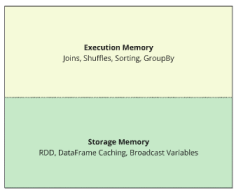
⚠ The two people get wrong: 300 MB comes off the user slice, not the 6 GB; overhead is added on top of the 10 GB. And Spark really subtracts the 300 MB before multiplying: 0.6 × (10240 − 300) = 5964 MB, not 6 GB — exact figures on the typed pages.

✍️ Reading it backwards (the useful direction)

- node has **64 GB**, you want **4 executors** ⟶ 16 GB each including overhead
- overhead ≈ 10% ⟶ solve `x + max(384, 0.1x) = 16` ⟶ `x ≈ 14.5 GB`, so ask for `--executor-memory 14g`
- that 14 GB buys you only **8.4 GB** of unified memory, and **4.2 GB** of execution space shared by every core on that executor
- 5 cores per executor ⟶ roughly **840 MB** of execution memory per concurrent task. That is the number that decides whether you spill.

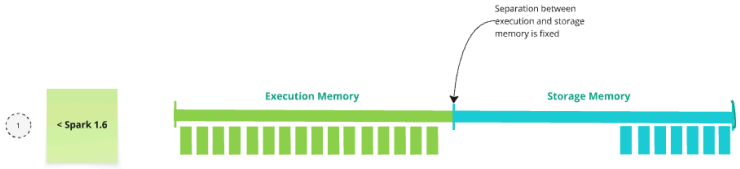
💡 Always divide execution memory by cores before claiming an executor is "big enough".

Unified memory — who gets to move the divider



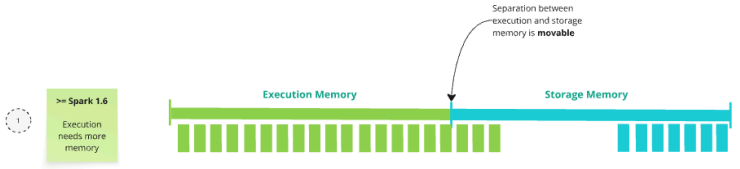
What "unified" means

- execution + storage are **one pool** = memory.fraction × executor-memory
- storageFraction only says where the divider starts, not where it stays
- user memory and reserved sit outside it and never move



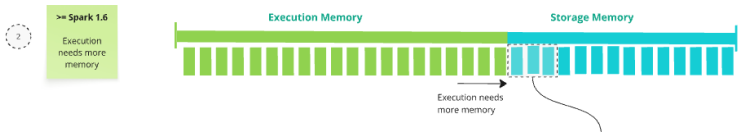
1 < Spark 1.6 — StaticMemoryManager

- the divider is FIXED
- execution could spill or OOM while storage sat half empty right next to it — pure waste
- you tuned the split by hand, per job



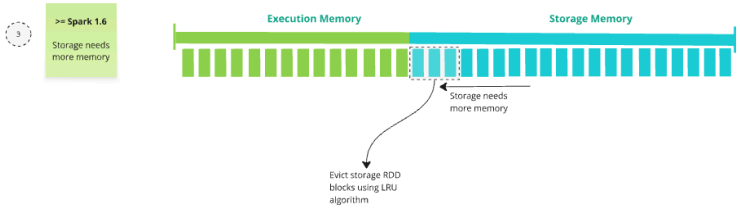
2 ≥ Spark 1.6 — UnifiedMemoryManager

- the divider is MOVABLE
- storage is not using its half, so execution takes the **vacant** space
- nothing is evicted here — nobody was there



3 Execution wants more, storage is full

- storage **evicts its own** blocks, LRU first, and hands the space over
- an evicted block is recomputed or read from disk later — you lose time, never correctness



4 Storage wants more, execution is full

- execution blocks are **never** evicted — a half-finished sort cannot be rebuilt from lineage
- storage evicts **its own** LRU blocks again, or the new block just does not get cached

The one rule

Execution always wins. Joins, shuffles, sorts and aggregations cannot be restarted halfway through from a lineage, so their memory is never taken away. Cached blocks can always be rebuilt, so they are the ones that go.

⚠ Gotcha: a `.cache()` on a DataFrame that gets read once does not merely waste storage memory — it pushes execution into spilling, and then gets evicted anyway. Cache only what is read twice or more.

💡 Storage borrows and gives it back; execution borrows and keeps it.

Execution vs Storage, side by side

Execution (wins)	Storage (yields)
joins, shuffles, sorts, groupBy buffers	cache() / persist() blocks, broadcast variables
lives for the length of one **task**	lives until evicted or unpersisted
cannot be rebuilt mid-flight → **never evicted**	always rebuildable from lineage → **first to go**
when it runs out: spill to disk, then OOM	when it runs out: silently drops blocks (Storage tab shows <100% cached)
knob: `memory.fraction`, and fewer `cores` per executor	knob: `storageFraction`, or cache less / cache serialized

Spark Memory Management — extra notes

Typed follow-ups to the annotated boards: the exact formula the board rounds off, off-heap memory in depth, what allocate/deallocate means in real code, and how the JVM garbage collector actually decides what to free.

1 · The exact formula (the board rounds, on purpose)

The video does `0.6 × 10 GB = 6 GB`. Spark does not. Reserved memory comes off **first**, and only what is left gets multiplied:

```
// UnifiedMemoryManager.getMaxMemory
usableMemory  = systemMemory - reservedMemory          // reservedMemory = 300 MB, hard-coded
maxMemory     = usableMemory * spark.memory.fraction   // the unified pool
onHeapStorage = maxMemory    * spark.memory.storageFraction
```

pool	board (teaching)	actual, for a 10 GiB heap
heap (`-Xmx`)	10 GB	10240 MB
reserved	300 MB	300 MB
usable	—	10240 − 300 = **9940 MB**
unified	0.6 × 10 = 6 GB	0.6 × 9940 = **5964 MB** (≈ 5.8 GiB)
storage (start)	3 GB	0.5 × 5964 = **2982 MB**
execution (start)	3 GB	**2982 MB**
user	4 GB − 300 MB = 3.7 GB	9940 − 5964 = **3976 MB**

Two things the simplified version hides. (a) Reserved is subtracted before the fraction, so you always get slightly *less* unified memory than `0.6 × heap`. (b) `Runtime.getRuntime.maxMemory` is itself a little below `-Xmx` (one survivor space is never usable), so the real pool is smaller again. Never size a job to the last hundred MB.

One-liner: the board's arithmetic is right to within ~1%; use it in an interview, but know that reserved comes off the top first.

2 · Every knob, with its default

property	default	what it controls
`spark.executor.memory`	1g	the JVM heap (`-Xmx`) of each executor — and nothing outside it
`spark.memory.fraction`	0.6	share of *(heap − 300 MB)* given to the unified pool. Lower it only if your own objects (user memory) are being squeezed.
`spark.memory.storageFraction`	0.5	where the execution/storage divider **starts**. It is also the floor: storage below this line cannot be evicted by execution.
reserved	300 MB	hard-coded in `UnifiedMemoryManager` ; no property exposes it
`spark.executor.memoryOverhead`	`max(384 MB, factor × executor.memory)`	everything in the process that is not the heap: metaspace, thread stacks, netty buffers, **Python workers**. Added on top of the heap when the container is requested.
`spark.executor.memoryOverheadFactor`	0.10	the factor above (Spark 3.3+). Non-JVM applications on Kubernetes default to 0.40 instead, because PySpark lives here.
`spark.memory.offHeap.enabled`	false	switch; on its own it does nothing
`spark.memory.offHeap.size`	0	the actual pool. **0 means off even when enabled is true.**
`spark.executor.pyspark.memory`	unset	caps each Python worker. Unset means bounded only by the container — the usual cause of a killed PySpark executor.
`spark.executor.cores`	1 / all, by manager	how many tasks share one executor's execution memory at once. This is the hidden divisor on every number above.

One-liner: only two of these are carved out of `executor.memory` (`fraction` and `storageFraction`); `memoryOverhead` and `offHeap.size` are added on top of it.

3 · Off-heap memory, properly

What "heap" even means

The **heap** is the single block of memory the JVM reserves at startup (`-Xmx`) and then hands out itself. Every `new` in Java or Scala carves a piece out of it, every object carries a 12–16 byte header so the JVM can find and describe it, and the **garbage collector owns the whole region** — it walks it, frees it, and moves objects around inside it.

Off-heap memory is memory the JVM asks the operating system for directly and then refuses to manage: a flat run of bytes at some address, with no object headers, no type information, and no GC. All that lives on the heap is a tiny handle — an 8-byte address and a length.

```
on-heap  Row  ->  [hdr|ptr]-->[hdr|ptr]-->[hdr|"ACME"]      3 objects, ~90 bytes, GC walks all 3
off-heap Row  ->  0x7f3a1c00: 04 00 00 00 41 43 4D 45 ...    1 blob,  ~12 bytes, GC sees nothing
```

Turning it on

```
spark.memory.offHeap.enabled   true      # default false
spark.memory.offHeap.size      2g        # default 0  -- BOTH are required
```

Setting only `enabled` gives you nothing: the size defaults to 0. Once on, the same `UnifiedMemoryManager` runs a **second** pair of pools — off-heap execution and off-heap storage — split by the same `storageFraction`. Off-heap is not a different feature; it is a second copy of the same picture, living outside the heap.

What actually goes there

goes off-heap	does NOT go off-heap
Tungsten execution buffers: sort records, the aggregation hash map, shuffle write buffers (`UnsafeRow` pages)	your Python / Scala objects, UDF closures, driver-side collections — those are user memory on the heap
blocks cached with `StorageLevel.OFF_HEAP`	JVM metaspace, thread stacks, netty network buffers — those come out of `memoryOverhead`

Why you would want it

- **No GC pressure.** The collector never scans it, so a 20 GB off-heap cache costs zero pause time. This is the real reason.
- **Denser.** No object headers and no pointer chasing — a 10-character Java `String` is ~56 bytes on heap, ~14 bytes as a Tungsten binary field.
- **Predictable.** The footprint is exactly what you asked for; it does not balloon and then shrink after a collection.

What it costs

- **Slower per access.** Every read and write crosses a serialize/deserialize boundary instead of dereferencing a pointer.
- **Invisible.** A leak never shows up in a heap dump, and `jmap` will tell you the heap is fine while the container dies.
- **Still counted by the OS.** YARN and Kubernetes measure the whole process, not the heap.

Sizing trap. Container = `executor.memory` + `memoryOverhead` + `offHeap.size` . Spark 3.x adds `offHeap.size` into the container request for you; on 2.x you had to inflate `memoryOverhead` by hand. Either way, turning off-heap on without giving the container more room just moves the OOM from the JVM to the kernel.

> *"I reach for off-heap when GC time on the Executors tab is eating more than ~10% of task time and the live set is large — typically a big cache or a wide aggregation. I start at 10–20% of executor memory, and I raise the container ask by the same amount. If spilling, not GC, is the problem, off-heap is the wrong lever; more execution memory is."*

One-liner: off-heap trades a little CPU per access for no GC pauses at all — and it is slower than the heap but orders of magnitude faster than spilling to disk, so it is the right choice when the alternative is disk.

4 · "You have to allocate and deallocate it yourself" — in code

True of the *mechanism*, mostly false of *your job*. Three levels:

Level 1 — on-heap: you allocate, the GC deallocates

```
byte[] buf = new byte[1024 * 1024];   // allocate: JVM carves 1 MB out of the heap
buf = null;                           // you only drop the reference...
                                      // ...the GC frees the bytes, whenever it feels like it
```

There is no `free()` in Java. That is the whole point of a managed heap — and the whole source of GC pauses.

Level 2 — off-heap, the semi-manual way (DirectByteBuffer)

```
ByteBuffer buf = ByteBuffer.allocateDirect(1024 * 1024);  // OS memory, outside -Xmx
buf.putLong(0, 42L);
long v = buf.getLong(0);
// Still no free(): the native bytes go back only when the small on-heap
// DirectByteBuffer object becomes unreachable and its Cleaner runs. Keep that
// object in a cache or a static map and the OS memory is never returned.
```

Capped by `-XX:MaxDirectMemorySize`; blowing that cap raises `java.lang.OutOfMemoryError: Direct buffer memory`.

Level 3 — off-heap, fully manual (what Spark itself does)

```
import org.apache.spark.unsafe.Platform;

long addr = Platform.allocateMemory(8L * 1024 * 1024);   // sun.misc.Unsafe.allocateMemory
try {
    Platform.putLong(null, addr, 42L);                   // write at a raw address
    long v = Platform.getLong(null, addr);
} finally {
    Platform.freeMemory(addr);   // <-- YOU call this. The GC never will, and if you
}                                //     miss it the memory is gone until the JVM exits.
```

What Spark wraps around that, so you do not have to

```
// TaskMemoryManager.allocatePage(size, consumer)
//   off-heap -> MemoryAllocator.UNSAFE.allocate(size)   == Unsafe.allocateMemory
//   on-heap  -> MemoryAllocator.HEAP.allocate(size)     == new long[...]
//
// Every page belongs to a MemoryConsumer (sorter, aggregate map, shuffle writer)
// and is registered against the task; at task end cleanUpAllAllocatedMemory()
// -> freePage() -> Platform.freeMemory(addr) for every page the task took.
```

So what is actually *your* job?

```
# 1. size it, and say so once
spark = (SparkSession.builder
         .config("spark.memory.offHeap.enabled", "true")
         .config("spark.memory.offHeap.size", "2g")
         .config("spark.executor.memoryOverhead", "1g")   # container must grow too
         .getOrCreate())
# 2. the only allocate/deallocate pair you write by hand:
from pyspark import StorageLevel
df.persist(StorageLevel.OFF_HEAP)   # allocate   -- blocks go to the off-heap store
df.unpersist()                      # deallocate -- or they sit there until the executor dies
```

✗ **What the video implies:** "with off-heap, the Spark developer has to allocate and deallocate memory."
✓ **More precisely:** writing DataFrame or SQL code, you never call `allocate` or `free` — `TaskMemoryManager` does it and releases every page when the task finishes. What really lands on you is: (a) **sizing** the pool and the container, (b) calling `unpersist()` on `OFF_HEAP` blocks, and (c) the fact that a leak here is **permanent and invisible**, because no GC is coming to clean up after you. You only write real `allocate` / `free` pairs if you implement your own `MemoryConsumer`, a custom data source, or a JNI/native UDF.

One-liner: on-heap, you forget and the GC forgives; off-heap, you forget and the kernel kills you.

5 · How the JVM knows an object is garbage ★

Reachability, not reference counts.

An object is garbage when no chain of references from a **GC root** reaches it — regardless of how many other objects point at it.

Your hypothesis, graded

"It goes through all the pointers you have in the program to see the memory they point to. Any memory nothing points to, it deletes."

- ✓ **Right in spirit.** Java GC is a **tracing** collector: it follows pointers and keeps whatever it can reach. Nothing else survives.
- ✗ **Correction 1 — direction.** It does not scan the heap looking for who points where. It starts from a small fixed set of **GC roots** and walks *outward*. Dead objects are never visited at all, which is why the cost of a collection is proportional to the **live** data, not to the garbage. Creating a million short-lived objects is nearly free; keeping a million alive is not.
- ✗ **Correction 2 — "pointed to" is not the test.** Two dead objects pointing at each other are still garbage. "Reachable from a root" is the test, and that is exactly the case reference counting gets wrong.

```
# island of isolation: both are "pointed to", both are still collected
a.next = b
b.prev = a
a = None
b = None        # no stack frame, no static field reaches them -> garbage
```

Worth knowing as a PySpark user: the two halves of your job use different schemes. **CPython** (the driver and the Python workers) uses **reference counting** — an object dies the instant its count hits zero — plus a cycle detector for exactly the island above. The **JVM** (every executor) uses tracing only. So "it's freed when nothing points at it" is the right mental model for your Python code and the wrong one for Spark's.

What counts as a GC root

- local variables and parameters in **every live stack frame** of **every thread**
- **static fields** of every loaded class (and the classes / classloaders themselves)
- JNI references held by native code
- objects currently used as a **monitor** (something is synchronized on them)
- a handful of JVM-internal references

That last-but-one bullet is why a `static Map` cache is the classic Java memory leak: a static field is a root, so everything it holds is live forever, by definition.

The actual algorithm: mark → sweep → compact

phase	what happens	cost driven by
Mark	Starting at the roots, follow every reference transitively and set a "live" bit on each object reached.	size of the **live set**
Sweep	Everything unmarked is, by definition, unreachable — no future line of code could name it. Its space is reclaimed.	heap size
Compact	Slide the survivors together so free space is one contiguous run and the next allocation is a pointer bump.	bytes of live data moved

Stop-the-world. Marking needs a reference graph that is not changing underneath it, so application threads are halted at safepoints while it runs (modern collectors do much of the marking concurrently, but never all of it). **That pause is the whole reason off-heap memory exists.**

"Java GC is a tracing collector. It starts from GC roots — thread stacks, statics, JNI handles — and marks everything transitively reachable; whatever is left unmarked is freed. It is not reference counting, so cyclic garbage is collected correctly, and the cost of a collection scales with the live set rather than with the amount of garbage."

6 · GC in practice: generations, G1, and why an executor stalls

The generational hypothesis

Nearly all objects die young. So the heap is split, and the cheap collection only looks at the part where that is true:

```
|---------------- Young ----------------|------------- Old (Tenured) -------------|
|   Eden      | S0 | S1 |               |   long-lived: cached blocks, broadcast  |
|  new objs   | survivors |              |   maps, the SparkContext, your statics  |

Minor GC : Eden + one survivor only. Copies the few survivors out, wipes the rest.
           Milliseconds. Runs constantly.
Promotion: survive N minor GCs -> moved to Old.
Major/Full GC: touches Old. Has to mark the entire live set. Seconds, on a big heap.
```

Why this bites a Spark executor specifically

- Shuffle and sort buffers are **short-lived and enormous** → they churn the young generation, which is fine.
- Cached DataFrames are **long-lived and enormous** → they fill the old generation with millions of live objects. Every full GC must mark all of them, every time. A 30 GB heap holding a 25 GB live cache is a multi-second pause with every core idle.
- **Three ways out**, in the order to try them:
  1. cache **serialized** (`MEMORY_AND_DISK_SER`) — one big `byte[]` instead of millions of objects, so the marker visits 1 object instead of 10^7;
  2. move the cache **off-heap** — not scanned at all;
  3. run **more, smaller executors** instead of one giant one — pause time tracks heap size, and 4×16 GB pauses far less than 1×64 GB.

Where to look

Spark UI → **Executors** tab → `GC Time` , next to `Task Time` . Rule of thumb: **GC Time above ~10% of Task Time means GC is your bottleneck**, not your logic.

Collectors worth knowing by name

collector	character	when
`ParallelGC`	multi-threaded, stop-the-world, best raw throughput, pauses grow with the heap	small heaps, batch jobs where a 2 s pause is irrelevant
`G1GC`	heap split into ~2000 regions, marks concurrently, collects the regions with most garbage first, aims at a *soft* pause target. **Default since Java 9.**	the sane default for executors above ~8 GB
`ZGC` / `Shenandoah`	nearly everything concurrent, sub-millisecond pauses, costs extra CPU and memory	very large heaps where latency matters more than throughput

```
--conf spark.executor.extraJavaOptions="-XX:+UseG1GC \
  -XX:MaxGCPauseMillis=200 \
  -Xlog:gc*:file=/tmp/gc-%p.log:time,uptime:filecount=5,filesize=20M"
```

> Never set `-Xmx` in `extraJavaOptions` — Spark sets it from `spark.executor.memory` and the duplicate will either be ignored or break the launch. And `System.gc()` is a request, not a command; in a Spark job it is almost always the wrong fix.

> **One-liner:** GC cost is paid per **live** byte, so the fix is never "collect more often" — it is "keep fewer live objects": serialize the cache, move it off-heap, or shrink the executor.

7 · Where does each thing live, and what breaks

region	holds	GC'd?	freed by	config
on-heap execution	shuffle / sort / agg buffers	yes	task completion	`memory.fraction`
on-heap storage	cached blocks, broadcasts	yes	LRU eviction, `unpersist()`	`memory.storageFraction`
user memory	your objects, UDF closures	yes	GC, when unreachable	whatever is left over
reserved	Spark internals	yes	—	300 MB, fixed
overhead	metaspace, thread stacks, netty buffers, **PySpark workers**	no (not heap)	OS / process exit	`executor.memoryOverhead`
off-heap	Tungsten pages, `OFF_HEAP` blocks	**no**	`TaskMemoryManager`, `unpersist()`	`memory.offHeap.size`
disk	spills, shuffle files, evicted blocks	n/a	shuffle cleanup / app end	`spark.local.dir`

Symptom → which knob

what you see	what it means	what to change
`Container killed by YARN for exceeding memory limits. 11.5 GB of 11 GB physical memory used` / K8s exit code 137	the **process** outgrew the container — almost always overhead, off-heap, or Python workers, not the heap	raise `executor.memoryOverhead`; if PySpark-heavy, cap `spark.executor.pyspark.memory`
`java.lang.OutOfMemoryError: Java heap space`	the heap genuinely filled — too much cached, or too many concurrent tasks sharing one heap	raise `executor.memory`, or lower `executor.cores`, or cache serialized
`OutOfMemoryError: Direct buffer memory`	off-heap / direct buffers exceeded their cap	`memory.offHeap.size` or `-XX:MaxDirectMemorySize`
Spill (memory) and Spill (disk) > 0 on the Stages tab	execution memory per task was too small for the partition	more shuffle partitions, fewer cores per executor, or fix the skew — see the data-skew notes
GC Time > 10% of Task Time	live set too large	`MEMORY_AND_DISK_SER`, off-heap, or smaller executors
Storage tab shows a cached DF at <100%	storage memory could not hold it; blocks are being evicted and recomputed	serialize it, or stop caching it

> **One-liner:** "killed" is a container problem, "OOM" is a heap problem, "spill" is an execution-memory problem, and "GC" is a live-set problem. They have four different fixes.

8 · Interview answers

"Walk me through what `--executor-memory 10G` buys you."

> *"10 GB is the JVM heap, nothing else. Spark takes 300 MB off the top as reserved, then `memory.fraction` 0.6 of the rest is the unified pool — about 5.8 GiB — which `storageFraction` 0.5 splits into roughly 2.9 GiB of execution and 2.9 GiB of storage to start with. The remaining ~3.9 GiB is user memory for my own objects. On top of the 10 GB, not inside it, Spark asks the cluster manager for `max(384 MB, 10%)` = 1 GB of overhead, so the container is 11 GB. If I enabled off-heap, that is a third amount again."*

"What is unified memory, and what changed in 1.6?"

> *"Before 1.6, `StaticMemoryManager` fixed the line between execution and storage, so a job could spill while half the storage pool sat empty. Since 1.6, `UnifiedMemoryManager` makes that line movable: either side can borrow the other's unused space. The asymmetry is that execution can force storage to evict LRU blocks to get memory back, but storage can never evict execution — a half-finished shuffle cannot be rebuilt from lineage, a cached block always can."*

"When would you turn off-heap on, and what is the catch?"

> *"When GC time, not spilling, is the bottleneck — a big cache or a wide aggregation on a large heap. It removes that memory from the collector's reach entirely. The catches are that every access pays a serialization cost, the container has to grow by the same amount or the kernel kills the executor, and a leak there never shows up in a heap dump."*

"How does the JVM know an object can be collected?"

> *"Reachability from GC roots — thread stacks, static fields, JNI handles, held monitors. The collector marks everything transitively reachable from those and frees the rest. It is tracing, not reference counting, so cycles of dead objects are collected correctly, and the cost scales with the live set rather than the garbage."*

"A job keeps dying with 'Container killed by YARN'. Walk me through it."

> *"That message is about the OS process, not the heap, so raising `executor.memory` usually makes it worse — it grows the container. I look at overhead first: netty shuffle buffers and, for PySpark, the Python worker processes all live there, and the 384 MB floor is far too small for a Python job. I'd raise `memoryOverhead` to 10–20% of executor memory, check whether off-heap is enabled and counted, and check whether one skewed partition is making a single task huge."*

9 · Cross-references and what is still open

Already covered in the other notes in this repo

- **Caching notes** — storage levels, `cache()` vs `persist()`, what eviction costs you, the Storage tab. The storage half of unified memory is explained there in detail; these notes do not repeat it.
- **Data-skew / salting / AQE notes** — spill is the symptom of execution memory running out on one fat partition; those notes have the measured runs.

Not covered yet

- `spark.executor.pyspark.memory` and how Python worker memory is accounted (it comes out of overhead, and is unbounded by default)
- Dynamic allocation and how it interacts with cached blocks on an executor that is about to be removed
- Tungsten internals: `UnsafeRow` layout, whole-stage codegen, and why they are what make off-heap worth having
- `spark.executor.cores` × execution memory — the real per-task budget, and how it drives `spark.sql.shuffle.partitions`

Next, in order

1. Executor tuning (the next video in the playlist) — it is the direct sequel: how many cores and how much memory per executor, and why 5 cores is the usual answer.
2. Run one job twice, once with `MEMORY_AND_DISK` and once with `MEMORY_AND_DISK_SER`, and read GC Time off the Executors tab. That single measurement makes section 6 permanent.
3. Then shuffle partitions, which is where the per-task execution budget actually gets spent.